Lista de Exercícios 01

Tratamento de Exceções

Cada aluno deverá, individualmente, resolver os exercícios abaixo:

Exercício 1 — Divisão com tratamento de erro

Desenvolva um programa que simule uma **calculadora simples de divisão**.

O programa deve:

1. Solicitar ao usuário dois números inteiros.
2. Realizar a divisão entre esses números.
3. Exibir o resultado da operação.

Entretanto, o programa deve utilizar o bloco `try-catch` para tratar o erro que ocorre quando o usuário tenta **dividir um número por zero**.

Caso isso aconteça, o programa deve:

- exibir uma mensagem informando que a divisão por zero não é permitida;
- continuar executando normalmente até o final do programa.

Exercício 2 — Acesso seguro a vetor

Crie um programa que trabalhe com um **vetor de números inteiros com 5 posições**.

O programa deve:

1. Criar o vetor.
2. Preencher o vetor com valores definidos no próprio código.
3. Solicitar ao usuário que informe um **índice do vetor** que deseja acessar.

4. Exibir o valor armazenado naquela posição.

Utilize **tratamento de exceção** para lidar com a situação em que o usuário informa um índice **fora dos limites do vetor**.

Caso isso aconteça, o programa deve informar que o índice é inválido.

Exercício 3 — Conversão de dados digitados

Desenvolva um programa que solicite ao usuário que digite um **número inteiro**.

O programa deve:

1. Ler o valor digitado como **texto**.
2. Converter esse valor para inteiro utilizando o método:

```
Integer.parseInt()
```

Entretanto, o programa deve tratar a situação em que o usuário digita:

- letras
- símbolos
- valores que não representam números

Exercício 4 — Uso do bloco finally

Crie um programa que solicite ao usuário dois números inteiros e realize a **divisão entre eles**.

O programa deve utilizar a estrutura: try-catch-finally

Requisitos:

1. O cálculo deve ocorrer dentro do bloco `try`.
2. Caso ocorra divisão por zero, o erro deve ser tratado no `catch`.

3. Independentemente de ocorrer erro ou não, o bloco `finally` deve exibir a mensagem:

Encerrando operação da calculadora.

Exercício 5 — Tratamento de múltiplas exceções

Crie um programa que execute duas operações diferentes:

1. Converter um número digitado pelo usuário para inteiro.
2. Armazenar esse número em uma posição de um vetor com 3 elementos.

O programa deve solicitar ao usuário:

- um número
- uma posição do vetor

Durante a execução podem ocorrer dois tipos de erro:

- o usuário digitar um valor que não é número
- o usuário informar uma posição inexistente no vetor

Utilize múltiplos blocos `catch` para tratar cada tipo de erro separadamente.

Exercício 6 — Validação de valor de produto com exceção lançada manualmente

Desenvolva um programa que simule o **cadastro do preço de um produto em um sistema de vendas**.

O programa deve executar as seguintes etapas:

1. Solicitar ao usuário que informe o **preço de um produto**.
2. Ler o valor digitado e armazená-lo em uma variável numérica.
3. Verificar se o valor informado é **válido**.

Considere as seguintes regras de validação:

- O preço do produto **não pode ser menor ou igual a zero**.
- Caso o usuário informe um valor inválido, o programa deve **lançar manualmente uma exceção** utilizando a instrução:

```
throw
```

A exceção deve informar que **o preço informado é inválido para cadastro de produto**.

Caso o valor seja válido, o programa deve exibir uma mensagem confirmando que o **produto foi cadastrado com sucesso**.

Requisitos adicionais:

- O lançamento da exceção deve ocorrer **no momento da validação do preço**.
- O tratamento da exceção deve ser realizado utilizando **bloco** **`try-catch`**.
- A mensagem exibida ao usuário deve explicar claramente o motivo do erro.

Exercício 7 — Método com declaração de exceção

Crie um programa com um método chamado:

```
converterNumero()
```

Esse método deve:

1. receber uma **String** como parâmetro
2. tentar converter essa String para um número inteiro
3. retornar o valor convertido

Caso ocorra erro na conversão, o método deve **declarar que pode lançar uma exceção** utilizando a palavra-chave:

```
throws
```

O tratamento da exceção deve ser realizado no método **main**.

Exercício 8 — Criação de exceção personalizada

Crie uma classe de exceção personalizada chamada:

`NumeroNegativoException`

Depois desenvolva um programa que solicite ao usuário um número inteiro.

O programa deve verificar a seguinte regra:

- o número digitado **não pode ser negativo**

Caso o usuário informe um número negativo, o programa deve:

- lançar a exceção personalizada criada
- exibir uma mensagem informando que números negativos não são permitidos.

Exercício 9 — Calculadora com validação de operação

Desenvolva uma calculadora simples que permita ao usuário escolher uma operação.

O programa deve exibir o seguinte menu:

1 - Soma
2 - Subtração
3 - Multiplicação
4 - Divisão

Depois de escolher a operação, o usuário deve informar dois números.

O programa deve tratar os seguintes erros:

- escolha de operação inválida
- divisão por zero
- digitação de valores não numéricos

Utilize **tratamento de exceções adequado para cada situação**.

Exercício 10 — Sistema de validação de dados

Desenvolva um programa que simule o **cadastro de um usuário**.

O programa deve solicitar:

- nome
- idade
- salário

O sistema deve validar as seguintes regras:

1. O nome não pode estar vazio.
2. A idade deve ser maior que zero.
3. O salário não pode ser negativo.

Caso alguma dessas regras seja violada, o programa deve lançar **exceções apropriadas** e informar ao usuário qual dado é inválido.

Exercício 11 — Leitura de dados com Scanner

Crie um programa que realize a **leitura de números digitados pelo usuário** e calcule a **média aritmética** desses valores.

O programa deve seguir os seguintes passos:

1. Utilizar um objeto da classe `Scanner` para realizar a leitura dos dados digitados pelo usuário.
2. O objeto `Scanner` deve ser declarado **dentro da estrutura** **`try-with-resources`**.
3. Solicitar ao usuário que informe **três números inteiros**.
4. Armazenar os valores informados em variáveis.
5. Calcular a média aritmética dos três números.
6. Exibir o resultado da média na tela.

Requisitos técnicos

O programa deve:

- utilizar a estrutura:

```
try (Scanner scanner = new Scanner(System.in)) {

}
```

- demonstrar que o recurso (`Scanner`) será **fechado automaticamente ao final do bloco `try`**.
- conter pelo menos um **bloco `catch`** para tratar possíveis erros de entrada de dados.

Exercício 12 — Processador de Tarefas

Parte 1 — Criação da classe do recurso

Crie a classe `ProcessadorTarefas` com as seguintes características:

1. A classe deve **implementar a interface `AutoCloseable`**.
2. Ao criar um objeto dessa classe, o sistema deve exibir a mensagem:

   `"Processador de tarefas iniciado."`

3. A classe deve possuir um método chamado:

   `executarTarefa(String tarefa)`

   Esse método deve exibir na tela:

   `"Executando tarefa: <nome da tarefa>"`

4. A classe deve implementar o método obrigatório da interface `AutoCloseable`: `close()`

Esse método deve exibir a mensagem:

```
"Processador de tarefas finalizado."
```

Parte 2 — Utilização do recurso com try-with-resources

Crie uma classe principal contendo o método `main`.

Nesse método, utilize a estrutura **`try-with-resources`** para criar um objeto da classe `ProcessadorTarefas`.

Dentro do bloco `try`, execute as seguintes tarefas:

1. `"Processar relatório"`
2. `"Atualizar dados"`
3. `"Gerar estatísticas"`

Cada tarefa deve ser executada utilizando o método `executarTarefa`.

Comportamento esperado

Quando o programa for executado, a sequência de mensagens deve indicar claramente:

1. início do recurso
2. execução das tarefas
3. finalização automática do recurso

Exemplo esperado de saída:

```
Processador de tarefas iniciado.
Executando tarefa: Processar relatório
Executando tarefa: Atualizar dados
Executando tarefa: Gerar estatísticas
Processador de tarefas finalizado.
```

Observe que a finalização ocorre **automaticamente quando o bloco `try` termina**.

Bom trabalho.